Lập trình mạng · Chương 5

Hiệu năng: NIO, Selector và mẫu Reactor

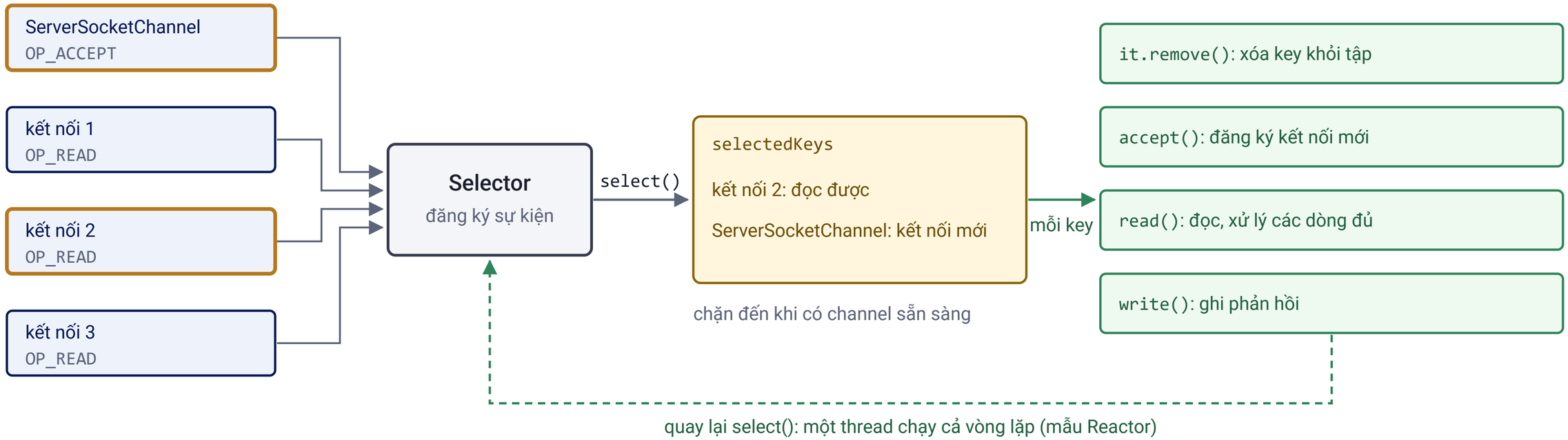
TS. Đặng Ngọc Hùng

Thread-per-connection: mỗi kết nối giữ một thread, kể cả lúc chỉ chờ

1. Thread-per-connection: 5 000 kết nối là 5 000 thread, phần lớn chỉ chờ dữ liệu
2. Mỗi platform thread cần **stack** riêng; hệ điều hành chuyển ngữ cảnh giữa các thread
3. **NIO**: một thread theo dõi nhiều kết nối, chỉ xử lý kết nối đã có dữ liệu

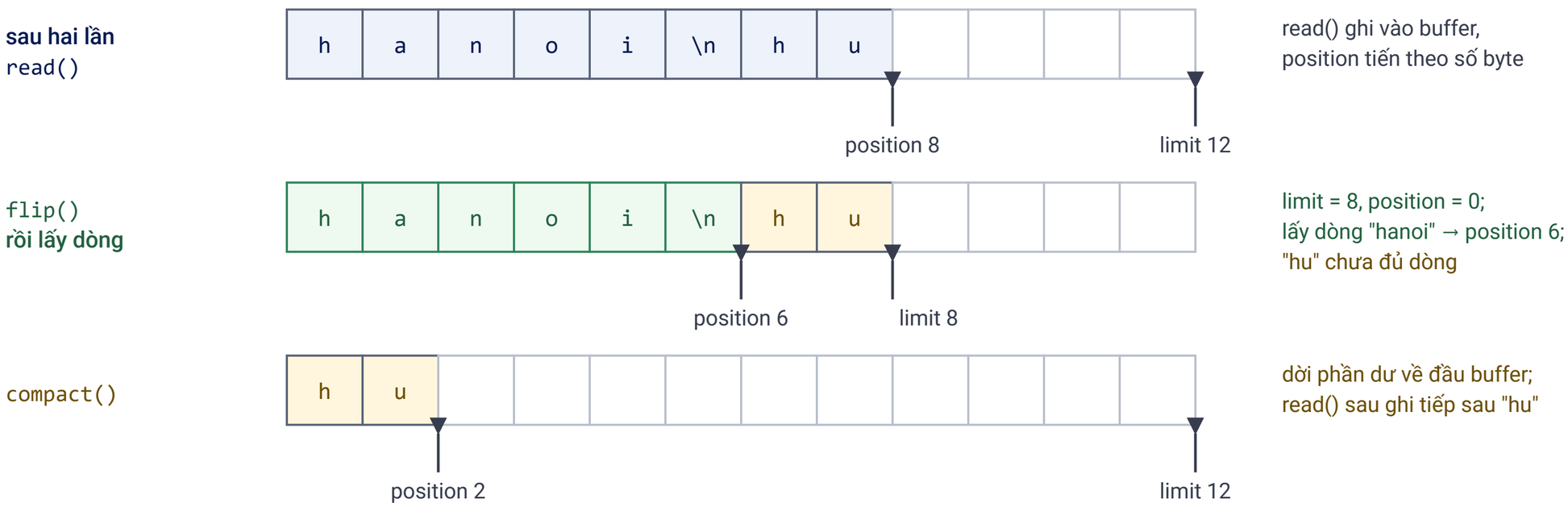
> **Câu hỏi của phần:** một thread phục vụ nhiều kết nối bằng cách nào, và đổi lại chương trình phải tự làm thêm gì?

Selector: một thread chờ sự kiện trên nhiều channel



Mẫu Reactor: một vòng lặp chờ sự kiện bằng `select()`, rồi chuyển từng **SelectionKey** sẵn sàng cho handler; không thao tác nào chặn.

ByteBuffer: flip() để đọc, compact() để giữ phần dòng chưa đủ



ByteBuffer của mỗi kết nối: `read()` ghi vào, `flip()` chuyển sang đọc, `compact()` giữ phần dòng chưa đủ cho lần đọc sau.

Vòng lặp Selector của server tra cứu

```java
listener.configureBlocking(false);  // accept() không chặn
listener.register(selector, SelectionKey.OP_ACCEPT);
while (true) {
    selector.select();  // chặn đến khi có ít nhất một channel sẵn sàng
    Iterator<SelectionKey> it = selector.selectedKeys().iterator();
    while (it.hasNext()) {
        SelectionKey key = it.next();
        it.remove();  // không xóa thì vòng sau key này bị xử lý lại
        if (key.isAcceptable()) {
            accept(listener, selector);
        } else if (key.isReadable()) {
            read(key);
```

- `configureBlocking(false)` rồi `register`: đăng ký sự kiện với Selector
- `it.remove()`: Selector không tự xóa key đã xử lý khỏi tập
- Một thread chạy cả vòng lặp: mọi handler phải trả về nhanh, không chặn

5 000 kết nối: số thread và thông lượng của ba cách

```java
ExecutorService workers = mode.equals("virtual")
        ? Executors.newVirtualThreadPerTaskExecutor()                    // mỗi kết nối một virtual thread
        : Executors.newThreadPerTaskExecutor(Thread.ofPlatform().factory());  // mỗi kết nối một platform thread
```

Kết quả chạy

```
nio       5000 open connections,     7 live platform threads
platform  5000 open connections,  5007 live platform threads
virtual   5000 open connections,    19 live platform threads
nio       18,177 requests/s   p50 250.9 ms   p99 461.0 ms   0 errors
platform  30,793 requests/s   p50 109.8 ms   p99 472.1 ms   0 errors
virtual   25,291 requests/s   p50 162.7 ms   p99 367.4 ms   0 errors
```

- NIO tiết kiệm **thread**, nhưng một vòng lặp chỉ dùng một lõi CPU
- Server thật chạy nhiều vòng lặp Selector; virtual thread dựa trên cùng cơ chế chờ sự kiện

Một lần read có thể chỉ nhận một phần dòng: ghép dòng trong buffer riêng

```java
int n = channel.read(c.in);  // không chặn: trả về số byte đang có, có thể chỉ là một phần dòng
c.in.flip();  // chuyển buffer từ ghi sang đọc: limit = position, position = 0
    if (c.in.get(i) == '\n') {  // đủ một dòng
c.in.compact();  // giữ phần dòng chưa đủ ở đầu buffer, chờ lần read sau
```

Kết quả chạy

```
nio          0 open connections,     7 live platform threads
read 3 bytes, 0 complete line(s), 3 byte(s) kept
read 5 bytes, 1 complete line(s), 2 byte(s) kept
read 4 bytes, 1 complete line(s), 0 byte(s) kept
```

- `read()` không chặn trả về số byte đang có: có thể chỉ là **một phần dòng**
- Mỗi kết nối một buffer riêng; đủ dòng rồi mới giải mã UTF-8 (framing)

Câu hỏi 1

Phương thức `select()` của Selector trả về 3. Điều đó có nghĩa gì?

A. Có 3 key vừa sẵn sàng cho sự kiện đã đăng ký; chúng nằm trong tập selectedKeys.

B. Server đang có đúng 3 kết nối mở.

C. Server vừa đọc được 3 byte.

D. Có 3 thread đang chờ trong `select()`.

Câu hỏi 1

Phương thức `select()` của Selector trả về 3. Điều đó có nghĩa gì?

A. Có 3 key vừa sẵn sàng cho sự kiện đã đăng ký; chúng nằm trong tập selectedKeys.

B. Server đang có đúng 3 kết nối mở.

C. Server vừa đọc được 3 byte.

D. Có 3 thread đang chờ trong `select()`.

`select()` trả về số key vừa sẵn sàng, không phải số kết nối, số byte hay số thread. Chương trình duyệt selectedKeys để xử lý từng key.

Câu hỏi 2

Một server NIO dùng chung một ByteBuffer cho mọi kết nối. Hai client cùng gửi yêu cầu, mỗi lần read chỉ nhận một phần dòng. Điều gì xảy ra?

A. Không sao, vì Selector xử lý từng kết nối một.

B. Phần dòng chưa đủ của hai kết nối trộn vào cùng buffer, server ghép sai dòng.

C. Selector tự tạo buffer riêng cho từng kết nối.

D. `read()` chặn cho đến khi đủ dòng, nên không có phần dòng nào.

Câu hỏi 2

Một server NIO dùng chung một ByteBuffer cho mọi kết nối. Hai client cùng gửi yêu cầu, mỗi lần read chỉ nhận một phần dòng. Điều gì xảy ra?

A. Không sao, vì Selector xử lý từng kết nối một.

B. Phần dòng chưa đủ của hai kết nối trộn vào cùng buffer, server ghép sai dòng.

C. Selector tự tạo buffer riêng cho từng kết nối.

D. `read()` chặn cho đến khi đủ dòng, nên không có phần dòng nào.

Phần dòng chưa đủ phải chờ lần đọc sau của đúng kết nối đó; dùng chung buffer thì byte của các kết nối trộn lẫn. Selector không quản lý buffer, và `read()` không chặn.

Tổng kết

1. **Non-blocking I/O** và **Selector**: một thread chờ sự kiện trên nhiều **SocketChannel** (**mẫu Reactor**)

2. **ByteBuffer** riêng cho mỗi kết nối: `flip()`, `compact()`, tự ghép đủ dòng

3. NIO tiết kiệm thread; **thông lượng** cần nhiều vòng lặp, **độ trễ** tăng khi yêu cầu xếp hàng

Từ khóa cần nhớ non-blocking I/O · SocketChannel · Selector · SelectionKey · ByteBuffer · mẫu Reactor · thông lượng · độ trễ

Mở rộng

Chủ đề nâng cao

- Netty: framework xây trên mẫu Reactor
- Mẫu Proactor và AsynchronousSocketChannel

Bài tập mở rộng

Chia kết nối cho nhiều Selector, mỗi lõi CPU một vòng lặp, rồi đo lại thông lượng bằng LoadClient.

Tiếp theo

Độ tin cậy: retry, request ID và checksum trên UDP